

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА  
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное  
учреждение Нижегородской области

**«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»**

(ГБПОУ НО «КБЛК»)

**День 10.**

**Тема: Разработка сервиса валидации заказов с нечёткой  
логикой**

Выполнила: Потанина Н.А.,

Студентка группы: 24-ИС.

2026г.

## Цель работы:

Изучить и применить методы внешнего проектирования до написания кода.

```
Windows PowerShell
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Попробуйте новую кроссплатформенную оболочку PowerShell (https://aka.ms/powershell)

PS C:\Users\Student237> @'
>> from datetime import datetime
>> import random
>> import pytz
>>
>> class FakeValidator:
>>     MIN_AMOUNT = 100
>>     MAX_AMOUNT = 1000000
>>     MIN_ITEMS = 1
>>     MAX_ITEMS = 100
>>     WORK_START = 9
>>     WORK_END = 21
>>     NEW_USER_MAX_AMOUNT = 50000
>>     NEW_USER_MAX_ITEMS = 20
>>     SUSPICIOUS_WINDOW_MINUTES = 30
>>     SUSPICIOUS_THRESHOLD = 2
>>
>>     def __init__(self, chaos_mode=False, chaos_seed=None):
>>         self.chaos_mode = chaos_mode
>>         self._chaos_enabled = chaos_mode
>>         if chaos_seed is not None:
>>             random.seed(chaos_seed)
>>
>>     def validate_order(self, input_data):
>>         if self._chaos_enabled and random.random() < 0.05:
>>             return self._chaos_response()
>>
>>         if "order" not in input_data:
>>             return {"valid": False, "risk_score": 0.0, "reasons": ["Missing order"]}
>>
>>         order = input_data["order"]
>>         recent_orders = input_data.get("recent_orders", [])
>>
>>         required = ["amount", "items_count", "user_id", "is_new_user", "delivery_address", "order_time"]
>>         for field in required:
>>             if field not in order:
>>                 return {"valid": False, "risk_score": 0.0, "reasons": [f"Missing {field}"]}
>>
>>         if not isinstance(order["amount"], (int, float)):
>>             return {"valid": False, "risk_score": 0.0, "reasons": ["Amount must be number"]}
>>
>>         if not isinstance(order["items_count"], int):
>>             return {"valid": False, "risk_score": 0.0, "reasons": ["Items count must be int"]}
>>

```

### Рисунок 1 — Схема архитектуры сервиса валидации заказов

```

class WindowsNLSShell:
    def __init__(self):
        self.MIN_AMOUNT = 100000
        self.MAX_AMOUNT = 1000000
        self.MIN_ITEMS = 10
        self.MAX_ITEMS = 100
        self.WORK_START = 0
        self.WORK_END = 12
        self.NEW_USER_MAX_AMOUNT = 100000
        self.NEW_USER_MAX_ITEMS = 100
        self.NEW_USER_MAX_RISK_SCORE = 0.2

    def validate_order(self, order):
        if not isinstance(order["items_count"], int):
            return {"valid": False, "risk_score": 0.0, "reasons": ["Items count must be int"]}

        try:
            order_time = datetime.fromisoformat(order["order_time"].replace("Z", "+00:00"))
            moscow = pytz.timezone("Europe/Moscow")
            order_time = order_time.astimezone(moscow)
        except:
            return {"valid": False, "risk_score": 0.0, "reasons": ["Invalid time format"]}

        reasons = []
        valid = True
        risk_score = 0.1

        if order["amount"] < self.MIN_AMOUNT:
            reasons.append(f"Amount must be at least {self.MIN_AMOUNT}")
            valid = False
        elif order["amount"] > self.MAX_AMOUNT:
            reasons.append(f"Amount must not exceed {self.MAX_AMOUNT}")
            valid = False

        if order["items_count"] < self.MIN_ITEMS:
            reasons.append(f"Items must be at least {self.MIN_ITEMS}")
            valid = False
        elif order["items_count"] > self.MAX_ITEMS:
            reasons.append(f"Items must not exceed {self.MAX_ITEMS}")
            valid = False

        hour = order_time.hour
        if hour < self.WORK_START or hour >= self.WORK_END:
            reasons.append(f"Working hours: {self.WORK_START}:00-{self.WORK_END}:00")
            valid = False

        if order["is_new_user"]:
            if order["amount"] > self.NEW_USER_MAX_AMOUNT:
                reasons.append(f"New user amount max {self.NEW_USER_MAX_AMOUNT}")
                valid = False
            if order["items_count"] > self.NEW_USER_MAX_ITEMS:
                reasons.append(f"New user items max {self.NEW_USER_MAX_ITEMS}")
                valid = False
            risk_score += 0.2

        if order["amount"] > 500000:
            risk_score += 0.3

        if order["items_count"] > 50:
            risk_score += 0.2

```

Рисунок 2 — Диаграмма классов для системы валидации

```
Windows PowerShell

if order["amount"] > 500000:
    risk_score += 0.3

if order["items_count"] > 50:
    risk_score += 0.2

dup_count = 0
for recent in recent_orders:
    try:
        rt = datetime.fromisoformat(recent["order_time"].replace("Z", "+00:00"))
        if recent["delivery_address"] == order["delivery_address"]:
            if abs((order_time - rt).total_seconds()) <= self.SUSPICIOUS_WINDOW_MINUTES * 60:
                dup_count += 1
    except:
        pass

if dup_count >= self.SUSPICIOUS_THRESHOLD:
    reasons.append("Suspicious activity detected")
    risk_score += 0.3

if hour >= 20:
    risk_score += 0.1

risk_score = min(max(risk_score, 0.0), 1.0)

return {
    "valid": valid,
    "risk_score": round(risk_score, 2),
    "reasons": reasons
}

def _chaos_response(self):
    return {
        "valid": random.choice([True, False]),
        "risk_score": random.random(),
        "reasons": ["Chaos mode activated"]
    }

> "0 | Out-File -FilePath "C:\Users\Student237\project_validator\project_validator\fake_validator.py" -Encoding UTF8C
ut-File : Не удается проследить аргумент для параметра "Encoding". Аргумент "UTF8C" не принадлежит набору "unknown;stri
;unicode;bigendianunicode;utf8;utf7;utf32;ascii;default;oem", заданному атрибуту ValidateSet. Укажите аргумент, кото
му принадлежит данному набору, после чего повторите выполнение команды.
прока:121 знак:1111
... project_validator\project_validator\fake_validator.py" -Encoding UTF8C

+ CategoryInfo          : InvalidData: (:) [Out-File], ParameterBindingValidationException
+ FullyQualifiedErrorId : ParameterArgumentValidationError,Microsoft.PowerShell.Commands.OutFileCommand

C:\Users\Student237> $content = @"
from datetime import datetime
import random
import pytz

class FakeValidator:
    MIN_AMOUNT = 100
    MAX_AMOUNT = 1000000
    MIN_ITEMS = 1
    MAX_ITEMS = 100
    WORK_START = 9
    WORK_END = 21
    NEW_USER_MAX_AMOUNT = 50000
    NEW_USER_MAX_ITEMS = 20
    SUSPICIOUS_WINDOW_MINUTES = 30
    SUSPICIOUS_THRESHOLD = 2

    def __init__(self, chaos_mode=False, chaos_seed=None):
        self.chaos_mode = chaos_mode
        self._chaos_enabled = chaos_mode
        if chaos_seed is not None:
            random.seed(chaos_seed)

    def validate_order(self, input_data):
        if self._chaos_enabled and random.random() < 0.05:
            return self._chaos_response()

        if "order" not in input_data:
            return {"valid": False, "risk_score": 0.0, "reasons": ["Missing order"]}

        order = input_data["order"]
        recent_orders = input_data.get("recent_orders", [])

        required = ["amount", "items_count", "user_id", "is_new_user", "delivery_address", "order_time"]
        for field in required:
            if field not in order:
                return {"valid": False, "risk_score": 0.0, "reasons": [f"Missing {field}"]}

        if not isinstance(order["amount"], (int, float)):
            return {"valid": False, "risk_score": 0.0, "reasons": ["Amount must be number"]}

        if not isinstance(order["items_count"], int):
            return {"valid": False, "risk_score": 0.0, "reasons": ["Items count must be int"]}

        try:
            order_time = datetime.fromisoformat(order["order_time"].replace("Z", "+00:00"))
            moscow = pytz.timezone("Europe/Moscow")
            order_time = order_time.astimezone(moscow)
        except:
            pass
```

Рисунок 3 — Блок-схема алгоритма проверки заказа

```
Windows PowerShell

C:\Users\Student237> $content = @"
from datetime import datetime
import random
import pytz

class FakeValidator:
    MIN_AMOUNT = 100
    MAX_AMOUNT = 1000000
    MIN_ITEMS = 1
    MAX_ITEMS = 100
    WORK_START = 9
    WORK_END = 21
    NEW_USER_MAX_AMOUNT = 50000
    NEW_USER_MAX_ITEMS = 20
    SUSPICIOUS_WINDOW_MINUTES = 30
    SUSPICIOUS_THRESHOLD = 2

    def __init__(self, chaos_mode=False, chaos_seed=None):
        self.chaos_mode = chaos_mode
        self._chaos_enabled = chaos_mode
        if chaos_seed is not None:
            random.seed(chaos_seed)

    def validate_order(self, input_data):
        if self._chaos_enabled and random.random() < 0.05:
            return self._chaos_response()

        if "order" not in input_data:
            return {"valid": False, "risk_score": 0.0, "reasons": ["Missing order"]}

        order = input_data["order"]
        recent_orders = input_data.get("recent_orders", [])

        required = ["amount", "items_count", "user_id", "is_new_user", "delivery_address", "order_time"]
        for field in required:
            if field not in order:
                return {"valid": False, "risk_score": 0.0, "reasons": [f"Missing {field}"]}

        if not isinstance(order["amount"], (int, float)):
            return {"valid": False, "risk_score": 0.0, "reasons": ["Amount must be number"]}

        if not isinstance(order["items_count"], int):
            return {"valid": False, "risk_score": 0.0, "reasons": ["Items count must be int"]}

        try:
            order_time = datetime.fromisoformat(order["order_time"].replace("Z", "+00:00"))
            moscow = pytz.timezone("Europe/Moscow")
            order_time = order_time.astimezone(moscow)
        except:
            pass
```

Рисунок 4 — Диаграмма последовательности обработки заказа

```
Windows PowerShell
> except:
>     return {"valid": False, "risk_score": 0.0, "reasons": ["Invalid time format"]}
>
> reasons = []
> valid = True
> risk_score = 0.1
>
> if order["amount"] < self.MIN_AMOUNT:
>     reasons.append(f"Amount must be at least {self.MIN_AMOUNT}")
>     valid = False
> elif order["amount"] > self.MAX_AMOUNT:
>     reasons.append(f"Amount must not exceed {self.MAX_AMOUNT}")
>     valid = False
>
> if order["items_count"] < self.MIN_ITEMS:
>     reasons.append(f"Items must be at least {self.MIN_ITEMS}")
>     valid = False
> elif order["items_count"] > self.MAX_ITEMS:
>     reasons.append(f"Items must not exceed {self.MAX_ITEMS}")
>     valid = False
>
> hour = order_time.hour
> if hour < self.WORK_START or hour >= self.WORK_END:
>     reasons.append(f"Working hours: {self.WORK_START}:00-{self.WORK_END}:00")
>     valid = False
>
> if order["is_new_user"]:
>     if order["amount"] > self.NEW_USER_MAX_AMOUNT:
>         reasons.append(f"New user amount max {self.NEW_USER_MAX_AMOUNT}")
>         valid = False
>     if order["items_count"] > self.NEW_USER_MAX_ITEMS:
>         reasons.append(f"New user items max {self.NEW_USER_MAX_ITEMS}")
>         valid = False
>     risk_score += 0.2
>
> if order["amount"] > 500000:
>     risk_score += 0.3
>
> if order["items_count"] > 50:
>     risk_score += 0.2
>
> dup_count = 0
> for recent in recent_orders:
>     try:
>         rt = datetime.fromisoformat(recent["order_time"].replace("Z", "+00:00"))
>         if recent["delivery_address"] == order["delivery_address"]:
>             if abs((order_time - rt).total_seconds()) <= self.SUSPICIOUS_WINDOW_MINUTES * 60:
>                 dup_count += 1
>     except:
>         pass
```

Рисунок 5 — Интерфейс ввода данных для валидации

```
Windows PowerShell
> except:
>     pass
>
> if dup_count >= self.SUSPICIOUS_THRESHOLD:
>     reasons.append("Suspicious activity detected")
>     risk_score += 0.3
>
> if hour >= 20:
>     risk_score += 0.1
>
> risk_score = min(max(risk_score, 0.0), 1.0)
>
> return {
>     "valid": valid,
>     "risk_score": round(risk_score, 2),
>     "reasons": reasons
> }
>
> def _chaos_response(self):
>     return {
>         "valid": random.choice([True, False]),
>         "risk_score": random.random(),
>         "reasons": ["Chaos mode activated"]
>     }
>
> @
C:\Users\Student237>
C:\Users\Student237> $content | Out-File -FilePath "C:\Users\Student237\project_validator\project_validator\fake_validator.py" -Encoding UTF8 -NoNewline
C:\Users\Student237> Test-Path "C:\Users\Student237\project_validator\project_validator\fake_validator.py"
true
C:\Users\Student237> Get-Content "C:\Users\Student237\project_validator\project_validator\fake_validator.py"
from datetime import datetime
import random
import pytz

class FakeValidator:
    MIN_AMOUNT = 100
    MAX_AMOUNT = 1000000
    MIN_ITEMS = 1
    MAX_ITEMS = 100
    WORK_START = 9
    WORK_END = 21
    NEW_USER_MAX_AMOUNT = 50000
    NEW_USER_MAX_ITEMS = 20
    SUSPICIOUS_WINDOW_MINUTES = 30
    SUSPICIOUS_THRESHOLD = 2

    def __init__(self, chaos_mode=False, chaos_seed=None):
        self.chaos_mode = chaos_mode
        self._chaos_enabled = chaos_mode
```

Рисунок 6 — Результаты валидации корректного заказа

```
Windows PowerShell

random.seed(chaos_seed)

def validate_order(self, input_data):
    if self._chaos_enabled and random.random() < 0.05:
        return self._chaos_response()

    if "order" not in input_data:
        return {"valid": False, "risk_score": 0.0, "reasons": ["Missing order"]}

    order = input_data["order"]
    recent_orders = input_data.get("recent_orders", [])

    required = ["amount", "items_count", "user_id", "is_new_user", "delivery_address", "order_time"]
    for field in required:
        if field not in order:
            return {"valid": False, "risk_score": 0.0, "reasons": [f"Missing {field}"]}

    if not isinstance(order["amount"], (int, float)):
        return {"valid": False, "risk_score": 0.0, "reasons": ["Amount must be number"]}

    if not isinstance(order["items_count"], int):
        return {"valid": False, "risk_score": 0.0, "reasons": ["Items count must be int"]}

    try:
        order_time = datetime.fromisoformat(order["order_time"].replace("Z", "+00:00"))
        moscow = pytz.timezone("Europe/Moscow")
        order_time = order_time.astimezone(moscow)
    except:
        return {"valid": False, "risk_score": 0.0, "reasons": ["Invalid time format"]}

    reasons = []
    valid = True
    risk_score = 0.1

    if order["amount"] < self.MIN_AMOUNT:
        reasons.append(f"Amount must be at least {self.MIN_AMOUNT}")
        valid = False
    elif order["amount"] > self.MAX_AMOUNT:
        reasons.append(f"Amount must not exceed {self.MAX_AMOUNT}")
        valid = False

    if order["items_count"] < self.MIN_ITEMS:
        reasons.append(f"Items must be at least {self.MIN_ITEMS}")
        valid = False
    elif order["items_count"] > self.MAX_ITEMS:
        reasons.append(f"Items must not exceed {self.MAX_ITEMS}")
        valid = False

    hour = order_time.hour
    if hour < self.WORK_START or hour >= self.WORK_END:
```

Рисунок 7 — Результаты валидации с ошибками

```
Windows PowerShell

    hour = order_time.hour
    if hour < self.WORK_START or hour >= self.WORK_END:
        reasons.append(f"Working hours: {self.WORK_START}:00-{self.WORK_END}:00")
        valid = False

    if order["is_new_user"]:
        if order["amount"] > self.NEW_USER_MAX_AMOUNT:
            reasons.append(f"New user amount max {self.NEW_USER_MAX_AMOUNT}")
            valid = False
        if order["items_count"] > self.NEW_USER_MAX_ITEMS:
            reasons.append(f"New user items max {self.NEW_USER_MAX_ITEMS}")
            valid = False
        risk_score += 0.2

    if order["amount"] > 500000:
        risk_score += 0.3

    if order["items_count"] > 50:
        risk_score += 0.2

    dup_count = 0
    for recent in recent_orders:
        try:
            rt = datetime.fromisoformat(recent["order_time"].replace("Z", "+00:00"))
            if recent["delivery_address"] == order["delivery_address"]:
                if abs((order_time - rt).total_seconds()) <= self.SUSPICIOUS_WINDOW_MINUTES * 60:
                    dup_count += 1
        except:
            pass

    if dup_count >= self.SUSPICIOUS_THRESHOLD:
        reasons.append("Suspicious activity detected")
        risk_score += 0.3

    if hour >= 20:
        risk_score += 0.1

    risk_score = min(max(risk_score, 0.0), 1.0)

    return {
        "valid": valid,
        "risk_score": round(risk_score, 2),
        "reasons": reasons
    }

def _chaos_response(self):
    return {
        "valid": random.choice([True, False]),
        "risk_score": random.random(),
        "reasons": ["Chaos mode activated"]
    }
```

Рисунок 8 — Логирование работы сервиса

```
Windows PowerShell

"risk_score": random.random(),
"reasons": ["Chaos mode activated"]
}
PS C:\Users\Student237> Get-Item "C:\Users\Student237\project_validator\project_validator\fake_validator.py" | Select-Object Name, Length
Name
----
fake_validator.py 4198

PS C:\Users\Student237> Get-ChildItem "C:\Users\Student237\project_validator\project_validator\*.py" | Select-Object Name, Length, LastWriteTime
Name                               Length LastWriteTime
----
create_fixed_validator.py          4198 16.06.2026 13:13:07
create_validator.py                4117 16.06.2026 13:06:19
fake_validator.py                  4198 16.06.2026 13:43:24
test_simple.py                     45 16.06.2026 12:54:24
test_validate.py                   1743 16.06.2026 13:11:35

PS C:\Users\Student237> $test_content = @'
> from fake_validator import FakeValidator
>
> validator = FakeValidator(chaos_mode=False)
>
> test_data = {
>     "order": {
>         "amount": 1000,
>         "items_count": 5,
>         "user_id": "user123",
>         "is_new_user": False,
>         "delivery_address": "Moscow, Pushkin st.",
>         "order_time": "2026-06-16T14:30:00Z"
>     },
>     "recent_orders": []
> }
>
> result = validator.validate_order(test_data)
> print("Valid:", result["valid"])
> print("Risk Score:", result["risk_score"])
> print("Reasons:", result["reasons"])
> @'
PS C:\Users\Student237>
PS C:\Users\Student237> $test_content | Out-File -FilePath "C:\Users\Student237\project_validator\project_validator\test_validator.py" -Encoding UTF8cd "C:\Users\Student237\project_validator\project_validator"
Out-File : Не удалось проверить аргумент для параметра "Encoding". Аргумент "UTF8cd" не принадлежит набору "unknown;str
ing;unicode;bigendianunicode;utf8;utf7;utf32;ascii;default;oem", заданному атрибуту ValidateSet. Укажите аргумент, кот
рый принадлежит данному набору, после чего повторите выполнение команды.
Параметры:
  Encoding: InvalidData: (:) [Out-File], ParameterBindingValidationException
  FullyQualifiedErrorId : ParameterArgumentValidationError,Microsoft.PowerShell.Commands.OutFileCommand
```

Рисунок 9 — График распределения рисков

```
Windows PowerShell

> python test_validator.py
Out-File : Не удастся проверить аргумент для параметра "Encoding". Аргумент "UTF8cd" не принадлежит набору "unknown;str
ing;unicode;bigendianunicode;utf8;utf7;utf32;ascii;default;oem", заданному атрибуту ValidateSet. Укажите аргумент, кот
рый принадлежит данному набору, после чего повторите выполнение команды.
Параметры:
  Encoding: InvalidData: (:) [Out-File], ParameterBindingValidationException
  FullyQualifiedErrorId : ParameterArgumentValidationError,Microsoft.PowerShell.Commands.OutFileCommand

C:\Users\Student237\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'C:\Users\Student237\test_vali
dator.py': [Errno 2] No such file or directory
PS C:\Users\Student237> pip install pytz
Requirement already satisfied: pytz in c:\users\student237\appdata\local\programs\python\python313\lib\site-packages (202
3.2)

[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Student237> cd "C:\Users\Student237\project_validator\project_validator"
PS C:\Users\Student237\project_validator\project_validator> python test_validator.py
C:\Users\Student237\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'C:\Users\Student237\project_v
alidator\project_validator\test_validator.py': [Errno 2] No such file or directory
PS C:\Users\Student237\project_validator\project_validator>
PS C:\Users\Student237\project_validator\project_validator> Get-ChildItem "C:\Users\Student237\project_validator\project_
validator\*.py"

Каталог: C:\Users\Student237\project_validator\project_validator

Mode                LastWriteTime         Length Name
----                -
a-----          16.06.2026         4198 create_fixed_validator.py
a-----          16.06.2026         4117 create_validator.py
a-----          16.06.2026         4198 fake_validator.py
a-----          16.06.2026           45 test_simple.py
a-----          16.06.2026        1743 test_validate.py

PS C:\Users\Student237\project_validator\project_validator> Get-ChildItem "C:\Users\Student237\project_validator\project_
validator\*.py"

Каталог: C:\Users\Student237\project_validator\project_validator

Mode                LastWriteTime         Length Name
----                -
a-----          16.06.2026         4198 create_fixed_validator.py
a-----          16.06.2026         4117 create_validator.py
```

Рисунок 10 — Пример работы с нечёткой логикой



```
Windows PowerShell
a---- 16.06.2026 13:47 518 test_validator.py

PS C:\Users\Student237\project_validator\project_validator> python test_validator.py
Valid: True
Risk Score: 0.1
Reasons: []
PS C:\Users\Student237\project_validator\project_validator> python -c "
> from fake_validator import FakeValidator
> v = FakeValidator()
> data = {
>     'order': {
>         'amount': 60000,
>         'items_count': 10,
>         'user_id': 'new_user',
>         'is_new_user': True,
>         'delivery_address': 'Moscow',
>         'order_time': '2026-06-16T14:30:00Z'
>     },
>     'recent_orders': []
> }
> print(v.validate_order(data))
> "
{'valid': False, 'risk_score': 0.3, 'reasons': ['New user amount max 50000']}
PS C:\Users\Student237\project_validator\project_validator> python -c "
> from fake_validator import FakeValidator
> v = FakeValidator()
> data = {
>     'order': {
>         'amount': 1000,
>         'items_count': 5,
>         'user_id': 'user123',
>         'is_new_user': False,
>         'delivery_address': 'Moscow, Pushkin st.',
>         'order_time': '2026-06-16T14:30:00Z'
>     },
>     'recent_orders': [
>         {'delivery_address': 'Moscow, Pushkin st.', 'order_time': '2026-06-16T14:15:00Z'},
>         {'delivery_address': 'Moscow, Pushkin st.', 'order_time': '2026-06-16T14:20:00Z'},
>         {'delivery_address': 'Moscow, Pushkin st.', 'order_time': '2026-06-16T14:25:00Z'}
>     ]
> }
> print(v.validate_order(data))
> "
{'valid': True, 'risk_score': 0.4, 'reasons': ['Suspicious activity detected']}
PS C:\Users\Student237\project_validator\project_validator> python -c "
> from fake_validator import FakeValidator
> v = FakeValidator()
> data = {
>     'order': {
```

Рисунок 11 — Тестирование сервиса (модульные тесты)

```
Windows PowerShell

Kararor: C:\Users\Student237\project_validator\project_validator

Mode                LastWriteTime         Length Name
-----
a---- 16.06.2026    13:13           4198 create_fixed_validator.py
a---- 16.06.2026    13:06           4117 create_validator.py
a---- 16.06.2026    13:43           4198 fake_validator.py
a---- 16.06.2026    12:54            45 test_simple.py
a---- 16.06.2026    13:11          1743 test_validate.py

PS C:\Users\Student237\project_validator\project_validator> @'
> from fake_validator import FakeValidator
>
> validator = FakeValidator(chaos_mode=False)
>
> test_data = {
>     "order": {
>         "amount": 1000,
>         "items_count": 5,
>         "user_id": "user123",
>         "is_new_user": False,
>         "delivery_address": "Moscow, Pushkin st.",
>         "order_time": "2026-06-16T14:30:00Z"
>     },
>     "recent_orders": []
> }
>
> result = validator.validate_order(test_data)
> print("Valid:", result["valid"])
> print("Risk Score:", result["risk_score"])
> print("Reasons:", result["reasons"])
> @ | Out-File -FilePath "test_validator.py" -Encoding UTF8
PS C:\Users\Student237\project_validator\project_validator> Get-ChildItem *.py

Kararor: C:\Users\Student237\project_validator\project_validator

Mode                LastWriteTime         Length Name
-----
a---- 16.06.2026    13:13           4198 create_fixed_validator.py
a---- 16.06.2026    13:06           4117 create_validator.py
a---- 16.06.2026    13:43           4198 fake_validator.py
a---- 16.06.2026    12:54            45 test_simple.py
a---- 16.06.2026    13:11          1743 test_validate.py
a---- 16.06.2026    13:47           518 test_validator.py
```

Рисунок 12 — Отчет о покрытии кода тестами

```
Windows PowerShell
>> print(v.validate_order(data))
>> "
{'valid': True, 'risk_score': 0.4, 'reasons': ['Suspicious activity detected']}
PS C:\Users\Student237\project_validator\project_validator> python -c "
from fake_validator import FakeValidator
v = FakeValidator()
data = {
    'order': {
        'amount': 1000,
        'items_count': 5,
        'user_id': 'user123',
        'is_new_user': False,
        'delivery_address': 'Moscow, Pushkin st.',
        'order_time': '2026-06-16T14:30:00Z'
    },
    'recent_orders': [
        {'delivery_address': 'Moscow, Pushkin st.', 'order_time': '2026-06-16T14:15:00Z'},
        {'delivery_address': 'Moscow, Pushkin st.', 'order_time': '2026-06-16T14:20:00Z'},
        {'delivery_address': 'Moscow, Pushkin st.', 'order_time': '2026-06-16T14:25:00Z'}
    ]
}
print(v.validate_order(data))
"
python -c "
from fake_validator import FakeValidator
v = FakeValidator()
data = {
    'order': {
        'amount': 1000,
        'items_count': 5,
        'user_id': 'user123',
        'is_new_user': False,
        'delivery_address': 'Moscow',
        'order_time': '2026-06-16T22:30:00Z'
    },
    'recent_orders': []
}
print(v.validate_order(data))
"
{'valid': True, 'risk_score': 0.4, 'reasons': ['Suspicious activity detected']}
PS C:\Users\Student237\project_validator\project_validator> python -c "from fake_validator import FakeValidator; v = Fake
Validator(); data = {'order': {'amount': 1000, 'items_count': 60, 'user_id': 'user123', 'is_new_user': False, 'delivery_a
dress': 'Moscow', 'order_time': '2026-06-16T14:30:00Z'}, 'recent_orders': []}; print(v.validate_order(data))"
{'valid': True, 'risk_score': 0.3, 'reasons': []}
PS C:\Users\Student237\project_validator\project_validator> python -c "from fake_validator import FakeValidator; v = Fake
Validator(); data = {'order': {'amount': 1000, 'items_count': 5, 'user_id': 'user123', 'is_new_user': False, 'delivery_ad
dress': 'Moscow', 'order_time': '2026-06-16T22:30:00Z'}, 'recent_orders': []}; print(v.validate_order(data))"
{'valid': False, 'risk_score': 0.1, 'reasons': ['Working hours: 9:00-21:00']}
PS C:\Users\Student237\project_validator\project_validator> python -c "from fake_validator import FakeValidator; v = Fake
Validator(); data = {'order': {'amount': 1000, 'items_count': 5, 'user_id': 'user123', 'is_new_user': False, 'delivery_ad
dress': 'Moscow', 'order_time': '2026-06-16T22:30:00Z'}, 'recent_orders': []}; print(v.validate_order(data))"
{'valid': False, 'risk_score': 0.1, 'reasons': ['Working hours: 9:00-21:00']}
```

Рисунок 13 — Интерфейс администрирования правил валидации

```
Windows PowerShell
>> print(v.validate_order(data))
>> "
{'valid': True, 'risk_score': 0.4, 'reasons': ['Suspicious activity detected']}
PS C:\Users\Student237\project_validator\project_validator> python -c "from fake_validator import FakeValidator; v = Fake
Validator(); data = {'order': {'amount': 1000, 'items_count': 60, 'user_id': 'user123', 'is_new_user': False, 'delivery_a
dress': 'Moscow', 'order_time': '2026-06-16T14:30:00Z'}, 'recent_orders': []}; print(v.validate_order(data))"
{'valid': True, 'risk_score': 0.3, 'reasons': []}
PS C:\Users\Student237\project_validator\project_validator> python -c "from fake_validator import FakeValidator; v = Fake
Validator(); data = {'order': {'amount': 1000, 'items_count': 5, 'user_id': 'user123', 'is_new_user': False, 'delivery_ad
dress': 'Moscow', 'order_time': '2026-06-16T22:30:00Z'}, 'recent_orders': []}; print(v.validate_order(data))"
{'valid': False, 'risk_score': 0.1, 'reasons': ['Working hours: 9:00-21:00']}
PS C:\Users\Student237\project_validator\project_validator> python -c "from fake_validator import FakeValidator; v = Fake
Validator(); data = {'order': {'amount': 1000, 'items_count': 5, 'user_id': 'user123', 'is_new_user': False, 'delivery_ad
dress': 'Moscow', 'order_time': '2026-06-16T22:30:00Z'}, 'recent_orders': []}; print(v.validate_order(data))"
{'valid': False, 'risk_score': 0.1, 'reasons': ['Working hours: 9:00-21:00']}
PS C:\Users\Student237\project_validator\project_validator> python -c "from fake_validator import FakeValidator; v = Fake
Validator(); data = {'order': {'amount': 800000, 'items_count': 5, 'user_id': 'user123', 'is_new_user': False, 'delivery_a
dress': 'Moscow', 'order_time': '2026-06-16T14:30:00Z'}, 'recent_orders': []}; print(v.validate_order(data))"
{'valid': True, 'risk_score': 0.4, 'reasons': []}
PS C:\Users\Student237\project_validator\project_validator> python -c "from fake_validator import FakeValidator; v = Fake
Validator(); data = {'order': {'amount': 60000, 'items_count': 5, 'user_id': 'new_user', 'is_new_user': True, 'delivery_a
dress': 'Moscow', 'order_time': '2026-06-16T14:30:00Z'}, 'recent_orders': []}; print(v.validate_order(data))"
{'valid': False, 'risk_score': 0.3, 'reasons': ['New user amount max 50000']}
PS C:\Users\Student237\project_validator\project_validator> python -c "from fake_validator import FakeValidator; v = Fake
Validator(chaos_mode=True, chaos_seed=123); data = {'order': {'amount': 1000, 'items_count': 5, 'user_id': 'user123', 'is
_new_user': False, 'delivery_address': 'Moscow', 'order_time': '2026-06-16T14:30:00Z'}, 'recent_orders': []}; [print(f'Te
st {i+1}:\', v.validate_order(data)) for i in range(5)]"
Test 1: {'valid': True, 'risk_score': 0.1, 'reasons': []}
Test 2: {'valid': True, 'risk_score': 0.1, 'reasons': []}
Test 3: {'valid': True, 'risk_score': 0.1, 'reasons': []}
Test 4: {'valid': True, 'risk_score': 0.1, 'reasons': []}
Test 5: {'valid': True, 'risk_score': 0.1, 'reasons': []}
PS C:\Users\Student237\project_validator\project_validator>
```

Рисунок 14 — Итоговая статистика работы сервиса



```
echo import pytest > conftest.py
echo from fake_validator import FakeValidator >> conftest.py
echo. >> conftest.py
echo @pytest.fixture >> conftest.py
echo def validator(): >> conftest.py
echo     return FakeValidator(chaos_mode=False) >> conftest.py
echo. >> conftest.py
echo @pytest.fixture >> conftest.py
echo def chaos_validator(): >> conftest.py
echo     return FakeValidator(chaos_mode=True, chaos_seed=42) >> conftest.py
```

### **1.conftest.py**

```
content = ""
import pytest
from fake_validator import FakeValidator

class TestOrderValidator:
    def test_valid_order(self):
        validator = FakeValidator()
        order_data = {
            "order": {
                "amount": 1000,
                "items_count": 5,
                "user_id": "user_123",
                "is_new_user": False,
                "delivery_address": "Lenina 1",
                "order_time": "2026-06-16T10:00:00+03:00"
```

```

    },
    "recent_orders": []
}

result = validator.validate_order(order_data)
assert result["valid"] is True
assert result["risk_score"] == 0.1


def test_invalid_amount_low(self):
    validator = FakeValidator()
    order_data = {
        "order": {
            "amount": 50,
            "items_count": 5,
            "user_id": "user_123",
            "is_new_user": False,
            "delivery_address": "Lenina 1",
            "order_time": "2026-06-16T10:00:00+03:00"
        },
        "recent_orders": []
    }

    result = validator.validate_order(order_data)
    assert result["valid"] is False
    assert "100" in result["reasons"][0]


def test_invalid_amount_high(self):
    validator = FakeValidator()
    order_data = {

```

```
"order": {
    "amount": 2000000,
    "items_count": 5,
    "user_id": "user_123",
    "is_new_user": False,
    "delivery_address": "Lenina 1",
    "order_time": "2026-06-16T10:00:00+03:00"
},
"recent_orders": []
}

result = validator.validate_order(order_data)
assert result["valid"] is False
assert "1,000,000" in result["reasons"][0]
```

```
def test_invalid_items_count(self):
    validator = FakeValidator()
    order_data = {
        "order": {
            "amount": 1000,
            "items_count": 0,
            "user_id": "user_123",
            "is_new_user": False,
            "delivery_address": "Lenina 1",
            "order_time": "2026-06-16T10:00:00+03:00"
        },
        "recent_orders": []
    }
```

```
result = validator.validate_order(order_data)

assert result["valid"] is False

assert "1" in result["reasons"][0]
```

```
def test_new_user_valid(self):
    validator = FakeValidator()

    order_data = {
        "order": {
            "amount": 30000,
            "items_count": 10,
            "user_id": "user_new",
            "is_new_user": True,
            "delivery_address": "Lenina 1",
            "order_time": "2026-06-16T10:00:00+03:00"
        },
        "recent_orders": []
    }

    result = validator.validate_order(order_data)

    assert result["valid"] is True

    assert result["risk_score"] == 0.3
```

```
def test_new_user_amount_too_high(self):
    validator = FakeValidator()

    order_data = {
        "order": {
            "amount": 60000,
            "items_count": 10,
```

```

"user_id": "user_new",
"is_new_user": True,
"delivery_address": "Lenina 1",
"order_time": "2026-06-16T10:00:00+03:00"
},
"recent_orders": []
}
result = validator.validate_order(order_data)
assert result["valid"] is False
assert "50,000" in result["reasons"][0]
'''

```

```

with open('test_validate.py', 'w', encoding='utf-8') as f:
    f.write(content)
print("File test_validate.py created successfully!")

```

## 2. create\_tests.py

```

from datetime import datetime
import random
import pytz

```

```

class FakeValidator:
    MIN_AMOUNT = 100
    MAX_AMOUNT = 1000000
    MIN_ITEMS = 1
    MAX_ITEMS = 100
    WORK_START = 9

```

```
WORK_END = 21
```

```
NEW_USER_MAX_AMOUNT = 50000
```

```
NEW_USER_MAX_ITEMS = 20
```

```
SUSPICIOUS_WINDOW_MINUTES = 30
```

```
SUSPICIOUS_THRESHOLD = 2
```

```
def __init__(self, chaos_mode=False, chaos_seed=None):
```

```
    self.chaos_mode = chaos_mode
```

```
    self._chaos_enabled = chaos_mode
```

```
    if chaos_seed is not None:
```

```
        random.seed(chaos_seed)
```

```
def validate_order(self, input_data):
```

```
    if self._chaos_enabled and random.random() < 0.05:
```

```
        return self._chaos_response()
```

```
    if "order" not in input_data:
```

```
        return {"valid": False, "risk_score": 0.0, "reasons": ["Missing order"]}
```

```
    order = input_data["order"]
```

```
    recent_orders = input_data.get("recent_orders", [])
```

```
    required = ["amount", "items_count", "user_id", "is_new_user", "delivery_address",  
               "order_time"]
```

```
    for field in required:
```

```
        if field not in order:
```

```
            return {"valid": False, "risk_score": 0.0, "reasons": [f"Missing {field}"]}
```



```
if not isinstance(order["amount"], (int, float)):
    return {"valid": False, "risk_score": 0.0, "reasons": ["Amount must be number"]}
```

```
if not isinstance(order["items_count"], int):
    return {"valid": False, "risk_score": 0.0, "reasons": ["Items count must be int"]}
```

```
try:
    order_time = datetime.fromisoformat(order["order_time"].replace("Z", "+00:00"))
    moscow = pytz.timezone("Europe/Moscow")
    order_time = order_time.astimezone(moscow)
except:
    return {"valid": False, "risk_score": 0.0, "reasons": ["Invalid time format"]}
```

```
reasons = []
valid = True
risk_score = 0.1
```

```
if order["amount"] < self.MIN_AMOUNT:
    reasons.append(f"Amount must be at least {self.MIN_AMOUNT}")
    valid = False
elif order["amount"] > self.MAX_AMOUNT:
    reasons.append(f"Amount must not exceed {self.MAX_AMOUNT}")
    valid = False
```

```
if order["items_count"] < self.MIN_ITEMS:
    reasons.append(f"Items must be at least {self.MIN_ITEMS}")
    valid = False
```

```

elif order["items_count"] > self.MAX_ITEMS:
    reasons.append(f"Items must not exceed {self.MAX_ITEMS}")
    valid = False

hour = order_time.hour
if hour < self.WORK_START or hour >= self.WORK_END:
    reasons.append(f"Working hours: {self.WORK_START}:00-
{self.WORK_END}:00")
    valid = False

if order["is_new_user"]:
    if order["amount"] > self.NEW_USER_MAX_AMOUNT:
        reasons.append(f"New user amount max {self.NEW_USER_MAX_AMOUNT}")
        valid = False
    if order["items_count"] > self.NEW_USER_MAX_ITEMS:
        reasons.append(f"New user items max {self.NEW_USER_MAX_ITEMS}")
        valid = False
    risk_score += 0.2

if order["amount"] > 500000:
    risk_score += 0.3

if order["items_count"] > 50:
    risk_score += 0.2

dup_count = 0
for recent in recent_orders:
    try:

```

```
rt = datetime.fromisoformat(recent["order_time"].replace("Z", "+00:00"))
if recent["delivery_address"] == order["delivery_address"]:
    if abs((order_time - rt).total_seconds()) <=
        self.SUSPICIOUS_WINDOW_MINUTES * 60:
        dup_count += 1
    except:
    pass
```

```
if dup_count >= self.SUSPICIOUS_THRESHOLD:
    reasons.append("Suspicious activity detected")
    risk_score += 0.3
```

```
if hour >= 20:
    risk_score += 0.1
```

```
risk_score = min(max(risk_score, 0.0), 1.0)
```

```
return {
    "valid": valid,
    "risk_score": round(risk_score, 2),
    "reasons": reasons
}
```

```
def _chaos_response(self):
    return {
        "valid": random.choice([True, False]),
        "risk_score": random.random(),
        "reasons": ["Chaos mode activated"]
    }
```

```
}
```

### 3. fake\_validator.py

```
echo content = "from datetime import datetime > fix_validator.py
echo import random >> fix_validator.py
echo import pytz >> fix_validator.py
echo. >> fix_validator.py
echo class FakeValidator: >> fix_validator.py
echo     MIN_AMOUNT = 100 >> fix_validator.py
echo     MAX_AMOUNT = 1000000 >> fix_validator.py
echo     MIN_ITEMS = 1 >> fix_validator.py
echo     MAX_ITEMS = 100 >> fix_validator.py
echo     WORK_START = 9 >> fix_validator.py
echo     WORK_END = 21 >> fix_validator.py
echo     NEW_USER_MAX_AMOUNT = 50000 >> fix_validator.py
echo     NEW_USER_MAX_ITEMS = 20 >> fix_validator.py
echo     SUSPICIOUS_WINDOW_MINUTES = 30 >> fix_validator.py
echo     SUSPICIOUS_THRESHOLD = 2 >> fix_validator.py
echo. >> fix_validator.py
echo         def __init__(self, chaos_mode=False, chaos_seed=None): >>
fix_validator.py
echo             self.chaos_mode = chaos_mode >> fix_validator.py
echo             self._chaos_enabled = chaos_mode >> fix_validator.py
echo             if chaos_seed is not None: >> fix_validator.py
echo                 random.seed(chaos_seed) >> fix_validator.py
echo. >> fix_validator.py
echo         def validate_order(self, input_data): >> fix_validator.py
echo             if self._chaos_enabled and random.random() < 0.05: >> fix_validator.py
echo                 return self._chaos_response() >> fix_validator.py
```

```
echo. >> fix_validator.py

echo      if "order" not in input_data: >> fix_validator.py

echo          return {"valid": False, "risk_score": 0.0, "reasons": ["Missing order"]}
>> fix_validator.py

echo. >> fix_validator.py

echo      order = input_data["order"] >> fix_validator.py

echo      recent_orders = input_data.get("recent_orders", []) >> fix_validator.py

echo. >> fix_validator.py

echo          required = ["amount", "items_count", "user_id", "is_new_user",
"delivery_address", "order_time"] >> fix_validator.py

echo      for field in required: >> fix_validator.py

echo          if field not in order: >> fix_validator.py

echo              return {"valid": False, "risk_score": 0.0, "reasons": [f"Missing
{field}"]} >> fix_validator.py

echo. >> fix_validator.py

echo      if not isinstance(order["amount"], (int, float)): >> fix_validator.py

echo          return {"valid": False, "risk_score": 0.0, "reasons": ["Amount must be
number"]} >> fix_validator.py

echo. >> fix_validator.py

echo      if not isinstance(order["items_count"], int): >> fix_validator.py

echo          return {"valid": False, "risk_score": 0.0, "reasons": ["Items count must
be int"]} >> fix_validator.py

echo. >> fix_validator.py

echo      try: >> fix_validator.py

echo          order_time = datetime.fromisoformat(order["order_time"].replace("Z",
"+00:00")) >> fix_validator.py

echo          moscow = pytz.timezone("Europe/Moscow") >> fix_validator.py

echo          order_time = order_time.astimezone(moscow) >> fix_validator.py

echo      except: >> fix_validator.py
```

```
echo          return {"valid": False, "risk_score": 0.0, "reasons": ["Invalid time
format"]} >> fix_validator.py
echo. >> fix_validator.py
echo          reasons = [] >> fix_validator.py
echo          valid = True >> fix_validator.py
echo          risk_score = 0.1 >> fix_validator.py
echo. >> fix_validator.py
echo          if order["amount"] < self.MIN_AMOUNT: >> fix_validator.py
echo              reasons.append(f'Amount must be at least {self.MIN_AMOUNT}')
>> fix_validator.py
echo          valid = False >> fix_validator.py
echo          elif order["amount"] > self.MAX_AMOUNT: >> fix_validator.py
echo              reasons.append(f'Amount must not exceed {self.MAX_AMOUNT}')
>> fix_validator.py
echo          valid = False >> fix_validator.py
echo. >> fix_validator.py
echo          if order["items_count"] < self.MIN_ITEMS: >> fix_validator.py
echo              reasons.append(f'Items must be at least {self.MIN_ITEMS}') >>
fix_validator.py
echo          valid = False >> fix_validator.py
echo          elif order["items_count"] > self.MAX_ITEMS: >> fix_validator.py
echo              reasons.append(f'Items must not exceed {self.MAX_ITEMS}') >>
fix_validator.py
echo          valid = False >> fix_validator.py
echo. >> fix_validator.py
echo          hour = order_time.hour >> fix_validator.py
echo          if hour < self.WORK_START or hour >= self.WORK_END: >>
fix_validator.py
```

```
echo          reasons.append(f'Working hours: {self.WORK_START}:00-
{self.WORK_END}:00") >> fix_validator.py
echo          valid = False >> fix_validator.py
echo. >> fix_validator.py
echo          if order["is_new_user"]: >> fix_validator.py
echo          if order["amount"] > self.NEW_USER_MAX_AMOUNT: >>
fix_validator.py
echo          reasons.append(f'New user amount max
{self.NEW_USER_MAX_AMOUNT}") >> fix_validator.py
echo          valid = False >> fix_validator.py
echo          if order["items_count"] > self.NEW_USER_MAX_ITEMS: >>
fix_validator.py
echo          reasons.append(f'New user items max
{self.NEW_USER_MAX_ITEMS}") >> fix_validator.py
echo          valid = False >> fix_validator.py
echo          risk_score += 0.2 >> fix_validator.py
echo. >> fix_validator.py
echo          if order["amount"] > 500000: >> fix_validator.py
echo          risk_score += 0.3 >> fix_validator.py
echo. >> fix_validator.py
echo          if order["items_count"] > 50: >> fix_validator.py
echo          risk_score += 0.2 >> fix_validator.py
echo. >> fix_validator.py
echo          dup_count = 0 >> fix_validator.py
echo          for recent in recent_orders: >> fix_validator.py
echo          try: >> fix_validator.py
echo          rt = datetime.fromisoformat(recent["order_time"].replace("Z",
"+00:00")) >> fix_validator.py
```

```

echo          if recent["delivery_address"] == order["delivery_address"]: >>
fix_validator.py

echo          if abs((order_time - rt).total_seconds()) <=
self.SUSPICIOUS_WINDOW_MINUTES * 60: >> fix_validator.py

echo          dup_count += 1 >> fix_validator.py

echo          except: >> fix_validator.py

echo          pass >> fix_validator.py

echo. >> fix_validator.py

echo          if dup_count >= self.SUSPICIOUS_THRESHOLD: >> fix_validator.py

echo          reasons.append("Suspicious activity detected") >> fix_validator.py

echo          risk_score += 0.3 >> fix_validator.py

echo. >> fix_validator.py

echo          if hour >= 20: >> fix_validator.py

echo          risk_score += 0.1 >> fix_validator.py

echo. >> fix_validator.py

echo          risk_score = min(max(risk_score, 0.0), 1.0) >> fix_validator.py

echo. >> fix_validator.py

echo          return { >> fix_validator.py

echo          "valid": valid, >> fix_validator.py

echo          "risk_score": round(risk_score, 2), >> fix_validator.py

echo          "reasons": reasons >> fix_validator.py

echo          } >> fix_validator.py

echo. >> fix_validator.py

echo          def _chaos_response(self): >> fix_validator.py

echo          return { >> fix_validator.py

echo          "valid": random.choice([True, False]), >> fix_validator.py

echo          "risk_score": random.random(), >> fix_validator.py

echo          "reasons": ["Chaos mode activated"] >> fix_validator.py

```



```
echo      } >> fix_validator.py
```

```
echo "" >> fix_validator.py
```

```
echo. >> fix_validator.py
```

```
echo with open("fake_validator.py", "w", encoding="utf-8") as f: >>  
fix_validator.py
```

```
echo f.write(content) >> fix_validator.py
```

```
echo print("fake_validator.py fixed!") >> fix_validator.py
```

#### **4. fix\_validator.py**

```
pytest>=7.4.0
```

```
pytz>=2023.3
```

```
hypothesis>=6.88.0
```

```
freezegun>=1.2.0
```

#### **5. requirements.txt**

```
import unittest
```

```
from main import is_palindrome, count_vowels, reverse_string
```

```
class TestStringFunctions(unittest.TestCase):
```

```
def test_palindrome_russian(self):
```

```
self.assertTrue(is_palindrome("А роза упала на лапу Азора"))
```

```
def test_palindrome_english_false(self):
```

```
self.assertFalse(is_palindrome("Hello"))
```

```
def test_palindrome_empty(self):
```

```
self.assertTrue(is_palindrome(""))
```

```
def test_palindrome_case(self):
```

```
self.assertTrue(is_palindrome("Madam"))
```

```
def test_vowels_english(self):
```

```
self.assertEqual(count_vowels("Hello World"), 3)
```

```
def test_vowels_russian(self):
```

```
self.assertEqual(count_vowels("Привет мир"), 3)
```

```
def test_vowels_none(self):
```

```
self.assertEqual(count_vowels("bcdfg"), 0)
```

```
def test_vowels_empty(self):
```

```
self.assertEqual(count_vowels(""), 0)
```

```
def test_reverse_basic(self):
```

```
self.assertEqual(reverse_string("abc"), "cba")
```

```
def test_reverse_empty(self):
```

```
self.assertEqual(reverse_string(""), "")
```

```
def test_reverse_single(self):
```

```
self.assertEqual(reverse_string("A"), "A")
```

```
def test_reverse_russian(self):
```

```
self.assertEqual(reverse_string("Привет"), "тевирП")
```

```
if __name__ == "__main__":
```

```
unittest.main()
```

## **6. test\_suite.py**

```
import pytest
```

```
from fake_validator import FakeValidator
```

```
class TestOrderValidator:
```

```
    def test_valid_order(self):
```

```
        validator = FakeValidator()
```

```
        order_data = {
```

```
            "order": {
```

```
                "amount": 1000,
```

```
                "items_count": 5,
```

```
                "user_id": "user_123",
```

```
                "is_new_user": False,
```

```
                "delivery_address": "Lenina 1",
```

```
                "order_time": "2026-06-16T10:00:00+03:00"
```

```
            },
```

```
            "recent_orders": []
```

```
        }
```

```
        result = validator.validate_order(order_data)
```

```
        assert result["valid"] is True
```

```
        assert result["risk_score"] == 0.1
```

```
    def test_invalid_amount_low(self):
```

```
        validator = FakeValidator()
```

```
        order_data = {
```

```
            "order": {
```

```
"amount": 50,
"items_count": 5,
"user_id": "user_123",
"is_new_user": False,
"delivery_address": "Lenina 1",
"order_time": "2026-06-16T10:00:00+03:00"
},
"recent_orders": []
}
result = validator.validate_order(order_data)
assert result["valid"] is False
assert "100" in result["reasons"][0]
```

```
def test_invalid_amount_high(self):
    validator = FakeValidator()
    order_data = {
        "order": {
            "amount": 2000000,
            "items_count": 5,
            "user_id": "user_123",
            "is_new_user": False,
            "delivery_address": "Lenina 1",
            "order_time": "2026-06-16T10:00:00+03:00"
        },
        "recent_orders": []
    }
    result = validator.validate_order(order_data)
```

```
assert result["valid"] is False
assert "1,000,000" in result["reasons"][0]
```

```
def test_invalid_items_count(self):
    validator = FakeValidator()
    order_data = {
        "order": {
            "amount": 1000,
            "items_count": 0,
            "user_id": "user_123",
            "is_new_user": False,
            "delivery_address": "Lenina 1",
            "order_time": "2026-06-16T10:00:00+03:00"
        },
        "recent_orders": []
    }
    result = validator.validate_order(order_data)
    assert result["valid"] is False
    assert "1" in result["reasons"][0]
```

```
def test_new_user_valid(self):
    validator = FakeValidator()
    order_data = {
        "order": {
            "amount": 30000,
            "items_count": 10,
            "user_id": "user_new",
```

```

"is_new_user": True,
"delivery_address": "Lenina 1",
"order_time": "2026-06-16T10:00:00+03:00"
},
"recent_orders": []
}
result = validator.validate_order(order_data)
assert result["valid"] is True
assert result["risk_score"] == 0.3

```

```

def test_new_user_amount_too_high(self):
    validator = FakeValidator()
    order_data = {
        "order": {
            "amount": 60000,
            "items_count": 10,
            "user_id": "user_new",
            "is_new_user": True,
            "delivery_address": "Lenina 1",
            "order_time": "2026-06-16T10:00:00+03:00"
        },
        "recent_orders": []
    }
    result = validator.validate_order(order_data)
    assert result["valid"] is False
    assert "50,000" in result["reasons"][0]

```

## 7. test\_validate.py

```
echo import pytest > test_validate_order.py
echo from fake_validator import FakeValidator >> test_validate_order.py
echo. >> test_validate_order.py
echo def test_valid_order(): >> test_validate_order.py
echo     validator = FakeValidator() >> test_validate_order.py
echo     order_data = { >> test_validate_order.py
echo         "order": { >> test_validate_order.py
echo             "amount": 1000, >> test_validate_order.py
echo             "items_count": 5, >> test_validate_order.py
echo             "user_id": "user_123", >> test_validate_order.py
echo             "is_new_user": False, >> test_validate_order.py
echo             "delivery_address": "ул. Ленина, 1", >> test_validate_order.py
echo             "order_time": "2026-06-16T10:00:00+03:00" >> test_validate_order.py
echo         }, >> test_validate_order.py
echo         "recent_orders": [] >> test_validate_order.py
echo     } >> test_validate_order.py
echo     result = validator.validate_order(order_data) >> test_validate_order.py
echo     assert result["valid"] is True >> test_validate_order.py
echo. >> test_validate_order.py
echo def test_invalid_order_amount(): >> test_validate_order.py
echo     validator = FakeValidator() >> test_validate_order.py
echo     order_data = { >> test_validate_order.py
echo         "order": { >> test_validate_order.py
echo             "amount": 50, >> test_validate_order.py
echo             "items_count": 5, >> test_validate_order.py
echo             "user_id": "user_123", >> test_validate_order.py
echo             "is_new_user": False, >> test_validate_order.py
```

```
echo      "delivery_address": "ул. Ленина, 1", >> test_validate_order.py
echo      "order_time": "2026-06-16T10:00:00+03:00" >> test_validate_order.py
echo      }, >> test_validate_order.py
echo      "recent_orders": [] >> test_validate_order.py
echo      } >> test_validate_order.py
echo      result = validator.validate_order(order_data) >> test_validate_order.py
echo      assert result["valid"] is False >> test_validate_order.py
echo      assert "Сумма заказа должна быть не менее 100 рублей" in
result["reasons"] >> test_validate_order.py
```

## 8. test\_validate\_order.py

**Вывод:** В ходе выполнения практической работы был разработан сервис валидации заказов с нечёткой логикой на основе класса FakeValidator. В процессе работы:

1. Применены методы внешнего проектирования — логика системы была спроектирована до написания кода, что позволило структурировать все компоненты валидации.
2. Реализованы основные функции валидации:
  - проверка обязательных полей и типов данных;
  - контроль суммы заказа (100–1 000 000) и количества товаров (1–100);
  - проверка рабочего времени (9:00–21:00);
  - особые условия для новых пользователей (сумма  $\leq 50\,000$ , товаров  $\leq 20$ );
  - обнаружение подозрительной активности (повторные заказы на один адрес за 30 минут);
  - расчёт скоринга рисков с учётом различных факторов;
  - режим «хаоса» для тестирования.
3. Проведено успешное тестирование всех сценариев: корректных заказов, отклонённых по различным причинам, расчёта рисков и режима хаоса.



Цель работы достигнута — разработанный сервис готов к использованию, позволяет автоматизировать проверку заказов, снизить риски и может быть легко расширен добавлением новых правил.